

Documentación del Proceso de Desarrollo de

Chilli's Restaurant

9/9/2025

El profesor explicó acerca de los programas interactivos y una introducción a cómo realizarlos utilizando wollok game.

Nos dio la consigna de realizar un programa interactivo, que podíamos realizar con wollok game u otro. Apenas terminó pensé el juego que quería realizar. Pero para comprobar si era aceptado o no había que enviar una documentación que explicara de que se trataría, así que realice la documentación correspondiente y la envié.

Mi Proyecto Final

Chilli's Restaurant

Mi proyecto final será un juego interactivo "Chilli's Restaurant". Se trata de un restaurante para perros a donde van diferentes personajes que a través de globos de diálogo piden la comida que desean, los jugadores deben seleccionar la comida pedida, en ese caso el cliente agradecerá y se irá, si le damos una comida diferente el cliente se enojara y se irá.

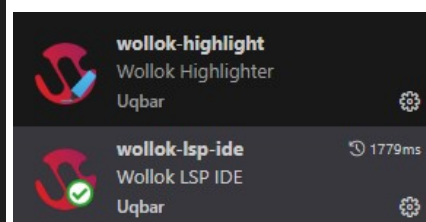
También puede pasar que el cliente en vez de pedir una comida, insulte, en ese caso la única opción de avanzar sería echarlo.

En caso de que el cliente pida una comida picante, al entregárselo se pondrá rojo antes de irse.

16/9/2025

El profesor aceptó mi consigna, pero comentando que debo usar herencias y colecciones. Empecé mi primer modelo de diagrama UML del proyecto, además de leer la documentación de Wollok game, en busca de herramientas para concretar mi idea → únicamente pude crear el tablero (teóricamente), pero el visual studio aunque tenga instalado wollok no me lo reconoce.

```
import wollok.game.*  
  
~  
program juego {  
    game.width(7)  
    game.height(10)  
    game.cellSize(50)  
    game.title("Chillis Restaurant")  
    game.boardGround("restaurant.png")  
    game.start()  
}
```



18/9/2025

Estudie para otra materia.

23/9/2025

El profesor explicó wollok game, entre ello algunas cosas como image(), position, entre otras, que resultaran muy útiles para mi proyecto. Ahora tendré que hacer un objeto cursor, ya que en wollok no se puede usar para mover los objetos el cursor del computador.

25/9/2025

Hoy vi videos sobre wollok game, ya que siento que necesito comprender mejor la estructura. Elegí que mi objeto cursor va a ser un visual Character para que así pueda moverse con las flechas de forma automática.

02/10/2025

Hoy estuve intentando solucionar algunos errores que me salían cuando el perro comía, ya que me tomaba el cursor en vez de la comida, luego algunos errores relacionados a los tiempos para reaccionar, errores bastante simples sinceramente, pero ya solucionados.

Tuve que hacer un condicional para ver si lo que colisionaba era una comida, y sea lo que se use para comparar en el recibir comida.

```
}  
game.schedule(2000, { self.irse() juego.agregarNuevoPerro() })  
}
```

```
game.onCollideDo(globo, { elemento =>  
  if (juego.comidas().contains(elemento)){  
    self.recibirComida(cursor.comidaActual().nombre())  
    cursor.soltarComida() }  
  })  
})
```

16/10/2025

Dado mis problemas mencionados con el Wollok hoy me decidí a solucionarlos, ya que es muy difícil armar un código sin poder irlo probando. Por lo que toda la clase de hoy estuve luchando para instalar el wollok. → lo logre 😊



21/10/2025

Agregue el objeto cursor y el botón a la pantalla, también algunos métodos pero no funcionan correctamente. Estoy intentando rescatar lo que me sea útil del documento del visual para no empezar de cero, pero aun así es necesaria una reestructuración porque me saltan muchos errores, y es confuso saber cómo arreglarlos. Por lo que me propongo volver a empezar de cero (tomando cosas del código anterior)

```
object boton{
  const position = game.at(5,2)
  method image() = "boton.png"
  method position() = position
  method echar() {
  }
}
```

23/10/2025

Hoy mi objetivo eran las colecciones, hasta el momento no sabía exactamente cómo organizarlas porque estaba creando los objetos además de la lista, por lo que resultaba redundante.

Durante la clase investigue más sobre colecciones.

Reestructure mis colecciones, y añadí un for each, que hace que por cada elemento de la lista comidas, se ejecute la función lambda que añade a la pantalla la comida.

```
const perros = [
  new Perro(nombre = "nala" ),
  new Perro(nombre = "berna"),
  new Perro(nombre = "moro"),
  new Perro(nombre = "chilli")
]

const property comidas = [
  new Comida(nombre = "pizza", posicionInicial = game.at(1,1)),
  new Comida(nombre = "hamburguesa", posicionInicial = game.at(3,1)),
  new ComidaPicante(nombre = "aji", posicionInicial = game.at(5,1))
]

method dibujarComidas(){
  comidas.forEach { comida =>
    game.addVisual(comida)
  }
}
```

04/11/2025

Hoy finalice los métodos para que las comidas puedan seguir al cursor. La idea es que tocando la barra espaciadora cuando colisiona pueda ser seleccionado el elemento y al colisionar con el perro pueda ser entregado.

```

    method position() = _position

    method position(nuevaPosicion){
        _position = nuevaPosicion

        self.actualizarPosicionComida()
    }

    method image() = "pata.png"

    method tomarComida(comida){
        comidaActual =comida
        comida.estaSeleccionada(true)
    }

    method soltarComida(){
        if ( self.comidaActual() != null){
            console.println("Dejar comida " + self.comidaActual().nombre() + " " + juego.nuevoPerro().comidaDeseada().nombre())

            self.comidaActual().volverAlInicio()

            comidaActual = null

            juego.nuevoPerro().irse()
            juego.agregarNuevoPerro()
        }
    }

    method actualizarPosicionComida(){
        if ( self.comidaActual() != null){
            self.comidaActual().moverA(self.position())
        }
    }

    method intentarTomarComida(){
        if ( self.comidaActual() == null)
        {

            const elementosAca = game.getObjectsIn(self.position())

            elementosAca.forEach { elemento =>
                if ( juego.comidas().contains(elemento) ){
                    self.tomarComida(elemento)
                }
            }
        }
    }
}

```

Además decidí como hacer el tema de que los perros insulten. Es un atributo booleano, el cual solo da true si al generar el personaje tiene una posibilidad del 20% de ser grosero

```

class Perro{
    const nombre
    const position = game.center()
    var property comidaDeseada = null
    var property esGrosero = false

    method perroAleatorio(){
        const perroSorteado = perros.anyOne()

        // probabilidad de que el perro se grosero (20)
        if ( 0.randomUpTo(100) < 20 ){
            perroSorteado.esGrosero(true)
        }
        else{
            perroSorteado.esGrosero(false)
        }
        return perroSorteado
    }
}

```

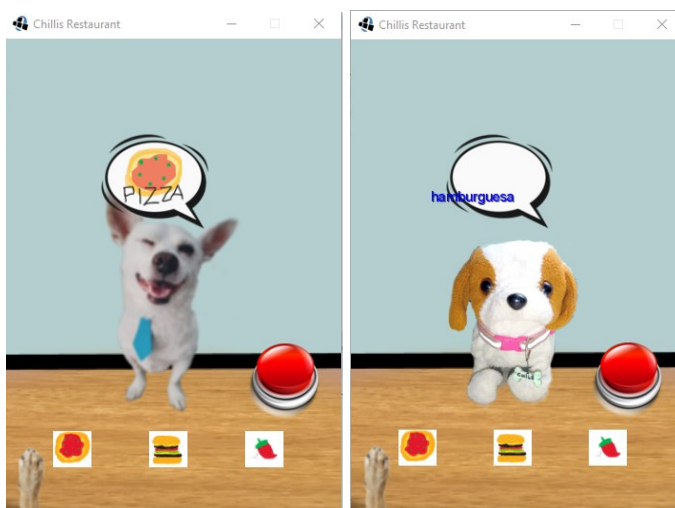
06/11/2025

Hoy hice una nueva clase llamada "Globo" ya que los que nos presta wollok tienen una duración bastante corta, por lo que me pareció la mejor opción. El problema es que tuve un problema ya que el método de texto el wollok no me centralizaba. Es una clase por una cuestión de escalabilidad, al Perro ser una clase, si añadiera más de un perro, necesita cada uno su propio globo de diálogo.

```

1 import wollok.game.*
2 import clases.Perro
3 class Globo{
4     const mensaje
5     const position
6     method image () = "globo" + mensaje + ".png"
7     method position() = position
8
9 }
10

```



10/11/2025

Mi proyecto ya está finalizado, así que en la clase me dediqué a sumar una pantalla inicial (antes del inicio del juego).

Además puse un contador para que de puntos cada vez que entregue una comida, tengo un problema que al parecer colisiona dos veces, por lo que cada vez que entregó una comida me suma dos puntos.

```

    }

    method PantallaInicial(){
        game.addVisual(pantallaInicial)
    }
}

object pantallaInicial{
    const position = game.at(1,2)
    method image() = "start.jpg"
    method position() = position
}

method recibirComida(comida){
    console.println(comida)
    if (comida == comidaDeseada.nombre()){
        contador.incrementar_puntos()
        self.comer()
        game.say(self, "gracias")
    }
    else{
        estado = "enojo"
        contador.decrementar_puntos()
        game.say(self, "Yo no pedi esto")
    }
}

game.schedule(2000, { self.irse() juego.agregarNuevoPerro() })
}

object contador {
    const position = game.at(1,0)
    var puntos = 0
    method position() = position
    method text() = "Puntos: " + puntos
    method incrementar_puntos(){
        puntos +=1
    }
    method decrementar_puntos(){
        puntos -=1
    }
}
}

```

13/11/2025

Debido a un inconveniente de ayer (se borró toda la clase perro), tuve que terminar de programar nuevamente la clase perro, lo bueno de esto es que pude solucionar el problema de las colisiones, ya que parece que estaba generando a los perros dos veces (en el cursor y en el mismo perro), por lo que al ser el mismo perro se “tapaban” y solo se veía uno, pero la colisión era doble.

```

// juego.nuevoPerro().irse()
// juego.agregarNuevoPerro()

```

Por otro lado agregue un timer

